

UNIVERSIDADE FEDERAL DE ALFENAS (UNIFAL-MG)

Bacharelado em Ciência da Computação

Disciplina: DCE792 – Algoritmos e Estruturas de Dados II (AEDS2)

Trabalho Prático 1
Saída do Labirinto

Alunos:	Gabriel de Souza Gonçalves, João Guilherme Couto
Professor:	Iago Augusto de Carvalho
Disciplina:	DCE792 – AEDS2
Data de entrega:	23/09/2026

1. Introdução

Este trabalho tem como objetivo implementar, em linguagem C, um programa capaz de encontrar um caminho entre a entrada e a saída de um labirinto representado por uma matriz de tamanho 10×10. Cada posição do labirinto é ocupada por um dos quatro símbolos: **E** (entrada), **S** (saída), **X** (parede) ou **0** (caminho livre). O problema consiste em, partindo da entrada, percorrer posições adjacentes — acima, abaixo, à esquerda ou à direita — sem atravessar paredes, até alcançar a saída, reportando ao final o caminho encontrado no formato *linha,coluna*.

O labirinto pode ser interpretado como um grafo não-direcionado, no qual cada casa livre é um vértice e existe uma aresta entre duas casas adjacentes que não sejam paredes. Encontrar um caminho entre a entrada e a saída equivale, portanto, a encontrar um caminho entre dois vértices desse grafo — problema clássico, normalmente resolvido por meio de busca em profundidade (DFS) ou busca em largura (BFS). Neste trabalho optou-se por uma busca em profundidade recursiva com retrocesso (*backtracking*), descrita na Seção 3.

Além do algoritmo de busca automática, o programa oferece um modo de jogo interativo, no qual o usuário controla manualmente um "jogador" dentro do labirinto usando as teclas W/A/S/D, e um modo tutorial explicativo. Um menu principal permite escolher entre os três labirintos de teste fornecidos (fácil, sem saída e difícil).

2. Estruturas de Dados

O labirinto não é armazenado como uma matriz convencional durante a execução do programa: ele é representado como uma estrutura celular ligada, em que cada casa é um nó que aponta diretamente para seus quatro vizinhos. Essa escolha permite que toda a navegação (impressão, jogo e busca da saída) seja feita apenas seguindo ponteiros, sem qualquer indexação por linha/coluna.

2.1. Célula do labirinto

```
typedef struct celula
{
    char tipo;                // 'E', 'S', 'X' ou '0'
    struct celula *cima;
    struct celula *baixo;
    struct celula *esquerda;
    struct celula *direita;

    struct celula *proximo;    // próxima célula do caminho encontrado
    int i;                    // linha original na matriz
    int j;                    // coluna original na matriz
    int visitado;              // marca de visita usada pela busca
} celula;
```

Os campos cima, baixo, esquerda e direita ligam cada célula diretamente às células vizinhas (ou apontam para NULL quando a vizinha estaria fora dos limites do labirinto), formando uma grade duplamente ligada. Os campos i e j guardam a posição original da célula na matriz, usados apenas para impressão do caminho. O campo proximo é preenchido pelo algoritmo de busca (Seção 3.2) para indicar qual vizinho faz parte do caminho até a saída, permitindo reconstruí-lo depois sem repetir a busca. Por fim, visitado é um marcador booleano que garante que a busca nunca processe a mesma célula mais de uma vez.

2.2. Labirinto

```
struct labirinto
{
    struct celula *entrada;
    struct celula *saida;
} *labirinto;
```

A estrutura labirinto mantém apenas dois ponteiros: para a célula de entrada e para a célula de saída. A partir do ponteiro entrada, todo o restante da grade é alcançável seguindo os ponteiros baixo e direita (por exemplo, é assim que imprimeLabirinto varre a matriz inteira sem nunca usar um índice).

Durante a construção (função criaLabirinto), utiliza-se uma matriz auxiliar local ponteiro[10][10] apenas para facilitar a ligação entre vizinhos por índice; essa matriz é descartada ao final da função e não faz parte da estrutura persistente do labirinto, que passa a existir somente como uma grade de células ligadas por ponteiros.

2.3. Organização em módulos (labirinto.h / labirinto.c)

O projeto foi dividido em um arquivo de cabeçalho, labirinto.h, e um arquivo de implementação, labirinto.c. O cabeçalho documenta, por meio de comentários no estilo Doxygen, cada campo das estruturas celula e labirinto, além do contrato (parâmetros e valor de retorno) de todas as funções do módulo: liberaMemoria, criaLabirinto, imprimeLabirintoComJogador, imprimeLabirinto, jogarLabirinto, mostrarCaminho, buscaSaida e mazeCrawler. Essa separação segue a prática usual em C de manter a interface pública de um módulo (o "o quê") isolada de sua implementação (o "como"), facilitando a leitura do projeto e a eventual reutilização das estruturas em outros arquivos-fonte.

3. Algoritmos

3.1. Construção do labirinto

A função criaLabirinto lê o arquivo de texto caractere a caractere, aloca uma célula para cada uma das 10×10 posições e, em seguida, percorre a grade novamente para ligar cada célula aos seus até quatro vizinhos. São dois laços aninhados independentes, cada um de tamanho fixo 10×10, portanto o custo é $O(R \times C)$, sendo R o número de linhas e C o número de colunas do labirinto (R = C = 10 neste trabalho).

3.2. Busca da saída — buscaSaida

O algoritmo central do trabalho é a função recursiva buscaSaida, uma busca em profundidade (DFS) com retrocesso:

```
int buscaSaida(celula *atual){
    if (atual == NULL || atual->tipo == 'X' || atual->visitado)
        return 0; // fora do labirinto, parede ou já visitada

    atual->visitado = 1; // marca permanente: nunca é desfeita

    if (atual->tipo == 'S')
        return 1; // encontrou a saída

    if (buscaSaida(atual->baixo)){
        atual->proximo = atual->baixo;
        if (atual->tipo != 'E') atual->tipo = '*';
    }
```

```

        return 1;
    }
    if (buscaSaida(atual->direita)){ ... }
    if (buscaSaida(atual->cima))    { ... }
    if (buscaSaida(atual->esquerda)){ ... }

    return 0;                      // beco sem saída
}

```

A cada chamada, a célula atual é imediatamente marcada como visitada, antes de qualquer tentativa nas quatro direções (baixo, direita, cima, esquerda, nessa ordem). O caso base rejeita células nulas (fora da grade), paredes ('X') ou já visitadas. Se a célula atual for a saída, a recursão retorna sucesso; caso contrário, o algoritmo tenta recursivamente cada vizinho e, assim que uma chamada recursiva retorna sucesso, a célula atual grava esse vizinho em *proximo* — efetivamente reconstruindo o caminho durante o desenrolar da recursão — e também retorna sucesso para quem a chamou. Se as quatro direções falharem, a função retorna 0, sinalizando um beco sem saída, sem desfazer a marcação de visitado.

3.2.1. Análise de complexidade

Como o campo *visitado* é definido de forma permanente — e é verificado no início da função, antes de qualquer trabalho ser feito —, cada uma das $R \times C$ células do labirinto é processada no máximo uma vez pela recursão. Em cada chamada, o trabalho realizado é constante: testar até quatro vizinhos e, no máximo, atualizar dois campos. Logo, o custo total é $O(R \times C)$, o que para o labirinto fixo 10×10 deste trabalho equivale a $O(100) = O(1)$, mas em geral, para um labirinto $n = R \times C$ células, a complexidade é $O(n)$.

Esse resultado é coerente com a complexidade clássica de uma busca em profundidade sobre um grafo, $O(V + E)$: tratando cada célula livre como um vértice e cada adjacência entre células livres como uma aresta, o grafo do labirinto tem grau no máximo 4, logo $E = O(4V) = O(V)$, e a expressão $O(V + E)$ se reduz a $O(V) = O(R \times C)$.

Vale destacar a importância de *visitado* nunca ser desfeito no retrocesso. Caso a marcação fosse temporária — desfeita sempre que uma direção levasse a um beco sem saída, para permitir que a mesma célula fosse reconsiderada por outro caminho —, a análise acima deixaria de valer: uma célula muito conectada poderia ser reprocessada do zero por diversos ramos diferentes da recursão. Em um labirinto com poucas paredes (alta conectividade), o número de chamadas recursivas cresceria exponencialmente com o número de células livres n , algo da ordem de $O(3^n)$ no pior caso (pois, ao chegar a uma célula, sobram até três direções válidas, excluindo a direção de onde se veio). É justamente a marcação permanente de visitado que garante que o algoritmo implementado seja, de fato, linear em $R \times C$.

3.3. Reconstrução e impressão do caminho — *mostrarCaminho*

Após uma busca bem-sucedida, *mostrarCaminho* parte da entrada e segue os ponteiros *proximo* gravados durante o retrocesso de *buscaSaida*, imprimindo cada posição no formato *linha,coluna* até alcançar a saída. Como cada célula do caminho é visitada uma única vez, o custo é $O(L)$, sendo L o comprimento do caminho encontrado ($L \leq R \times C$), ou seja, $O(R \times C)$ no pior caso.

3.4. Liberação de memória — *liberaMemoria*

Percorre a grade linha a linha (seguindo *baixo*) e, em cada linha, coluna a coluna (seguindo *direita*), liberando cada célula exatamente uma vez. Complexidade $O(R \times C)$.

3.5. Modo jogador — jogarLabirinto

No modo interativo, cada movimento do jogador é $O(1)$ — apenas segue um ponteiro de vizinho e verifica se é uma parede. A cada movimento, porém, o labirinto inteiro é reimpresso (`imprimeLabirintoComJogador`), o que custa $O(R \times C)$ por chamada. O custo total do modo jogo depende do número de jogadas realizadas pelo usuário, não sendo, portanto, uma medida de desempenho do algoritmo em si.

3.6. Complexidade total do programa

Para uma execução completa do modo de resolução automática (opção 3 do menu), o custo total é a soma das etapas: carregar o labirinto ($O(R \times C)$), buscar a saída ($O(R \times C)$) e reconstruir/imprimir o caminho ($O(R \times C)$). Como a soma de termos $O(R \times C)$ continua sendo $O(R \times C)$, a complexidade final do programa, em função do tamanho do labirinto, é **$O(R \times C)$** — linear no número de células.

4. Casos de Teste e Resultados

O algoritmo foi validado com as três instâncias de labirinto 10×10 fornecidas para teste. As figuras abaixo ilustram o resultado da execução da opção 3 do menu (resolução automática) sobre cada uma delas: em verde a entrada, em vermelho a saída, em cinza-escuro as paredes e em laranja as células que compõem o caminho encontrado pela buscaSaída.

4.1. Labirinto fácil



Instância bem conectada, em que o algoritmo encontra rapidamente um caminho válido entre a entrada e a saída.

4.2. Labirinto difícil



Instância com mais paredes e corredores estreitos; o caminho encontrado é mais longo, mas o algoritmo continua concluindo a busca em $O(R \times C)$.

4.3. Labirinto sem saída



Instância em que a saída ("S") está isolada por paredes. A função buscaSaída visita todas as células alcançáveis a partir da entrada (nenhuma é destacada em laranja) e corretamente retorna 0, e o programa informa que nenhum caminho válido foi encontrado.

5. Makefile e Compilação

A compilação do projeto é automatizada pelo Makefile a seguir, que compila separadamente cada arquivo `.c` encontrado no diretório em um objeto `.o` e, em seguida, os liga (*link*) no executável final `labirinto`:

```
# Nome do projeto
PROJ_NAME = labirinto

# Arquivos .c
C_SOURCE = $(wildcard *.c)

# Arquivos .h
H_SOURCE = $(wildcard *.h)

# Arquivos objeto
OBJ = $(C_SOURCE:.c=.o)

# Compilador
CC = gcc

# Opções do compilador
CC_FLAGS = -Wall -Wextra -g -pedantic

# Compilação
all: $(PROJ_NAME)

# Linkagem
$(PROJ_NAME): $(OBJ)
    $(CC) -o $@ $^

# Compilação dos arquivos .c em .o
%.o: %.c
    $(CC) $(CC_FLAGS) -c $< -o $@

# Limpeza
clean:
    rm -f *.o $(PROJ_NAME) *~
```

PROJ_NAME define o nome do executável gerado. **C_SOURCE** e **H_SOURCE** usam a função `wildcard` do `make` para descobrir automaticamente todos os arquivos `.c` e `.h` do diretório, dispensando listá-los manualmente caso o projeto cresça. **OBJ** deriva, a partir de **C_SOURCE**, o nome de cada objeto correspondente (trocando a extensão `.c` por `.o`). **CC_FLAGS** habilita avisos adicionais do compilador (`-Wall -Wextra -pedantic`, que ajudam a identificar erros comuns e desvios do padrão ISO C ainda em tempo de compilação) e mantém símbolos de depuração (`-g`).

A regra `all` (executada por padrão) depende do executável `labirinto`, que por sua vez depende da lista de objetos **OBJ**: a regra de linkagem `$(CC) -o $@ $^` une todos os `.o` no executável final. A regra padrão implícita `%.o: %.c` compila cada arquivo-fonte individualmente, de modo que, ao alterar apenas um arquivo `.c`, somente ele precisa ser recompilado antes de uma nova linkagem. Por fim, `clean` remove todos os objetos, o executável e arquivos de backup (`*~`).

5.1. Instruções de compilação e execução

```
# compilar (gera labirinto.o e o executável labirinto)
make

# executar (a partir do diretório que contém o executável e os
# arquivos .txt dos labirintos de teste)
./labirinto

# remover os arquivos .o e o executável gerados
```

`make clean`

É importante que os arquivos `labirinto_facil.txt`, `labirinto_sem_saida.txt` e `labirinto_dificil.txt` estejam no mesmo diretório de onde o executável é chamado, pois o programa os abre por caminho relativo (opção 4 do menu principal).